Разбор игры 2048.c и попытка ее сломать

Данные моей платформы (архитектура и версии предполагаемо используемых утилит для исследования):

Linux ARM64

```
parallels@panda:~$ uname -m
aarch64
parallels@panda:~$ git --version
git version 2.43.0
parallels@panda:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

parallels@panda:~$ gdb --version
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Рис. 1. Данные платформы

https://github.com/mevdschee/2048.c - репозиторий GitHub с нашей игрой.

Клонируем репозиторий в удобную нам папку, затем компилируем исполняемый файл используя компилятор gcc и параметры -O0 (для отключения оптимизации), -g0 (чтобы не было отладочной информации):

```
parallels@panda:~/Desktop/re-lab/re-writeups$ git clone https://github.com/mevdschee/2048.c
Cloning into '2048.c'...
remote: Enumerating objects: 576, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (40/40), done.
remote: Total 576 (delta 99), reused 91 (delta 83), pack-reused 453 (from 3)
Receiving objects: 100% (576/576), 125.37 KiB | 1.53 MiB/s, done.
Resolving deltas: 100% (313/313), done.
parallels@panda:~/Desktop/re-lab/re-writeups$ ls
2048.c
parallels@panda:~/Desktop/re-lab/re-writeups$ cd 2048.c
parallels@panda:~/Desktop/re-lab/re-writeups/2048.c$ ls
2048.c  2048.desktop  debian_2048.png  LICENSE  Makefile  palette.sh  README_es.md  README.md  screenshot.png
parallels@panda:~/Desktop/re-lab/re-writeups/2048.c$ gcc -O0 -g0 2048.c -o game
parallels@panda:~/Desktop/re-lab/re-writeups/2048.c$ file game
game: ELF 64-bit LSB pie executable, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-aarch64.so.1, Buil
dID[sha1]=3f27a8b3badf0f91419690655fd472f88c5ac8b9, for GNU/Linux 3.7.0, not stripped
```

Рис. 2. Клонирование репозитория

Также заодно я проверяю file game, чтобы увидеть архитектуру и формат исполняемого файла (далее я буду называть его бинарником/бинарем).

Запускаю игру:

```
./game
```

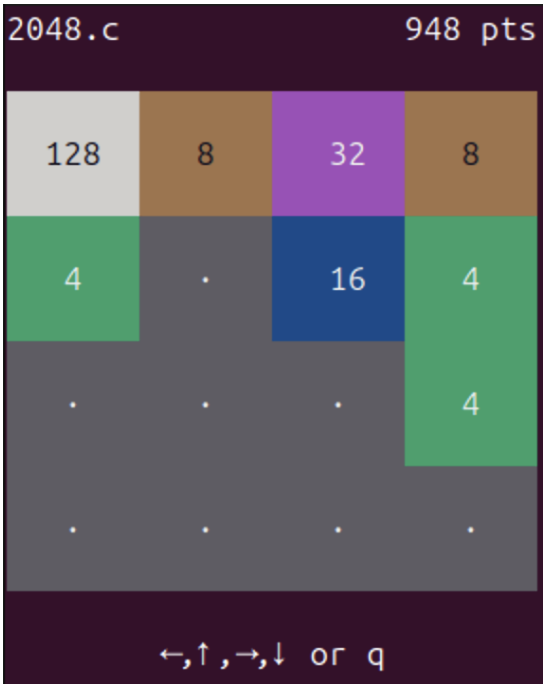


Рис. 3. Вид игры

Игра рабочая ☺

Исследование игры (бинарника) я решил начать с банального статического анализа строк утилитой strings -n 4, чтобы просмотреть файл и найти читаемые последовательности символов (текст, имена файлов/функций, названия библиотек…). Так как бинарник не отчищен от имен функций (not stripped), то можно найти что-то из интересного и выдвинуть предположения:

- time + srand@GLIBC_2.17 (вероятно для генерации квадратов 2 или 4 при каждом новом ходе игрока)
- --help, который вывел, как можно, с какими параметрами/модами запускать игру (например bluered: Use a blue-to-red color scheme или blackwhite: The black-to-white color scheme)

- “Error! Cannot read keyboard input”, “GAME OVER QUIT? (y/n) RESTART? (y/n)” - строчки, вызванные из-за неправильного пользовательского ввода и при проигрыше игры соответственно
- moveRight (сдвинуть квадраты вправо), moveLeft (сдвинуть квадраты влево), moveDown (сдвинуть квадраты вниз), moveUp (сдвинуть квадраты вверх)
- findTarget (?)
- countEmpty (?)
- findPairDown (?)
- drawBoard (функция отвечающая за отрисовку игровой доски)
- getColors (?)
- addRandom (функция отвечающая за появление случайного квадрата при каждом новом ходе игрока)
- getDigitCount (определить количество цифр в числе)
- gameEnded (условие для концовки игры)

Также можно проверить файл с помощью утилиты nm (с параметром -n для сортировки по адресу по возрастанию), которая покажет символы в файле и «флаг», который указывает на тип символа - T/U/t и другие. Это нужно для того, чтобы понять, реализована ли функция в данном файле или она ожидает найти какую-либо функцию во внешних библиотеках. К примеру:

```
0000000000001ea4 T main
```

Длинное число указывает на адрес (в 16й системе счисления) функции, main - название функции, а T - реализация этой функции содержится в исследуемом файле.

Завершить нашу первичную разведку ELF-файла мы можем с помощью readelf:

```
readelf -S game
```

Это нужно в случае, если мы хотим посмотреть расположение/размеры секций, если нужно исследовать конкретные данные. Но сейчас, я предполагаю, что можно обойтись и без этого.

Дальше исследование я буду проводить с помощью Ghidra - супер-полезная вещь для реверсеров (Описание с GitHub: Ghidra is a software reverse engineering framework created and maintained by the National Security Agency Research Directorate. This framework includes a suite of full-featured, high-end software analysis tools that enable users to analyze compiled code on a variety of platforms including Windows, macOS, and Linux.)

Так как бинарник у нас non-stripped, то в Symbol Tree очень легко искать функции игры:

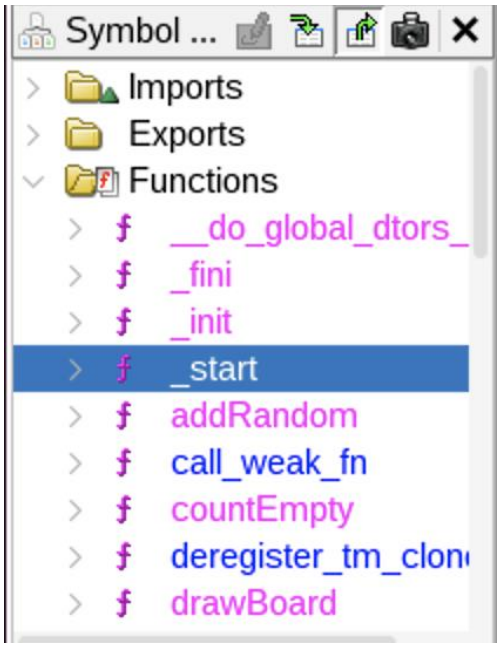


Рис. 4. Symbol Tree

Начну свое расследование с _start: попадая в нее, смотря на декомпилер (или же «псевдокод»), мы можем обратиться к главной функции - main:

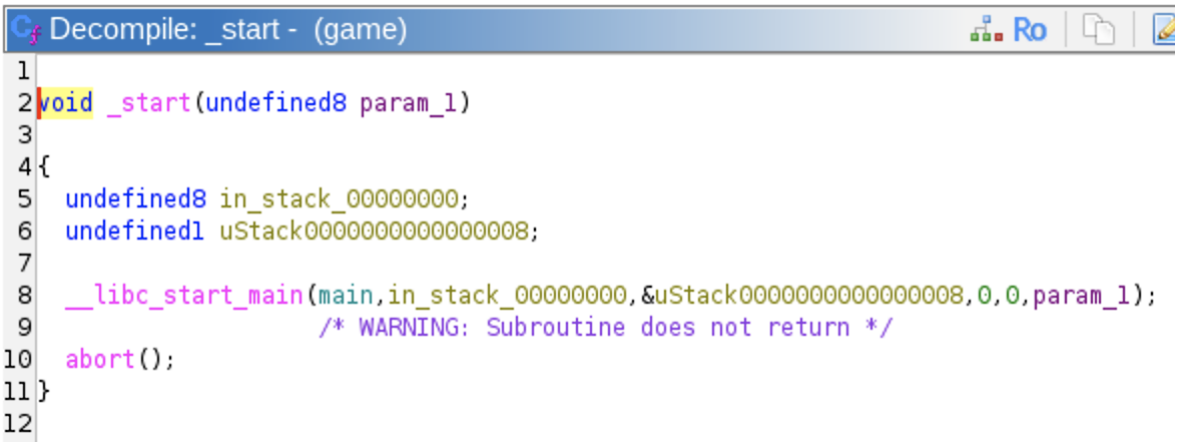


Рис. 5. Псевдокод функции _start

Начало псевдокода main у нас такое:



Рис. 6. Функция main

Т.к. param_1 вероятнее всего отвечает за количество аргументов (argc) с пользовательского ввода, то строчка 21: if (1 < param_1) {… отвечает за обработку аргументов по типу -h и т.п. Нас интересует переход к самой игре в случае 1 аргумента:

```
69    printf("\x1b[?25l\x1b[2J");
70    __sysv_signal(2,signal_callback_handler);
71    initBoard(auStack_28);
72    setBufferedInput(0);
73    drawBoard(auStack_28,local_32,local_30);
74    while (local_2c = getchar(), local_2c != -1) {
75      if (local_2c == 0x77) goto LAB_001022e4;
76      if (local_2c < 0x78) {
77        if (local_2c == 0x73) goto LAB_001022f8;
78        if (0x73 < local_2c) goto LAB_0010230c;
79        if (local_2c == 0x6c) goto LAB_001022d0;
80        if (0x6c < local_2c) goto LAB_0010230c;
81        if (local_2c == 0x6b) goto LAB_001022e4;
82        if (0x6b < local_2c) goto LAB_0010230c;
83        if (local_2c == 0x6a) {
84    LAB_001022f8:
85          local_31 = moveDown(auStack_28,&local_30);
```

Рис. 7. Отрывок функции main

Можем примерно предположить, что local_2 = getchar() считывает ввод человека в ходе игры (стрелочка вниз/вверх…) и ставит соответствия: если считался символ 0x73, то переход к функции LAB_001022f8: moveDown, или же действие - стрелочка вниз - сдвинуть квадраты вниз. Это, конечно, здорово, но игру сломать мне это, наверное, не поможет. Я хочу попробовать 2 способа изменить игру:

1) Поменять тотал «2048» на, условные, «128» (так выиграть игру будет гораздо проще)
2) Изменить арифметику игры

Начнем с попытки поменять «2048» на «128». Первое и очевидное, что мы можем сделать - попробовать найти константу 2048 или 0x800 (если в 16-сс).



Рис. 8. Поиск константы 0x800

Поиск ничего нам не дал. Тогда можно предположить, что игра считает плитки не как числа «2», «4», «8», а как степени двойки, то есть «1», «2», «3» и т.д. И, покопавшись в функциях, я понял: это предположение можно подтвердить.

```
35      else {
36        local_1c = 1 << (ulong)(*(byte *)(param_1 + (ulong)local_1f * 4 + (long)(int)(uint)local_1e)
37                                & 0x1f);
38        cVar1 = getDigitCount(local_1c);
39        local_1d = 7 - cVar1;
```

Рис. 9. Часть псевдокода drawBoard

Мы можем наблюдать побитовый сдвиг влево! Мы читаем один байт из игрового поля, а затем вычисляем степень двойки через побитовый сдвиг (на сколько единиц n мы сдвигаем влево = домножить число на 2^n).

Но как же происходит само сложение квадратов, то есть чисел? Из функций у нас есть только findPairDown, из названия можно предположить, что мы рассматриваем пару квадрат + квадрат снизу, то есть мы проверяем соседние квадраты относительно вертикали: текущий квадрат и квадрат под ним. И это предположение подтверждает псевдокод:

```
2 undefined8 findPairDown(long param_1)
3
4 {
5   byte local_3;
6   byte local_2;
7
8   local_3 = 0;
9   do {
10    if (3 < local_3) {
11      return 0;
12    }
13    for (local_2 = 0; local_2 < 3; local_2 = local_2 + 1) {
14      if (*(char *)(param_1 + (ulong)local_3 * 4 + (long)(int)(uint)local_2) ==
15          *(char *)(param_1 + (ulong)local_3 * 4 + (long)(int)(local_2 + 1))) {
16        return 1;
17      }
18    }
19    local_3 = local_3 + 1;
20  } while( true );
21 }
```

Рис. 10. findPairDown

А где функции для сложения квадратов, например, по горизонтали? Интересное наблюдение - их нет. Для сложения у нас есть только функция

findPairDown, подтверждающая или опровергающая, одинаковы ли соседние квадраты или нет, а в других случаях - когда мы рассматриваем сложение НЕ для квадрата под текущим квадратом, мы «крутим» поле с помощью функции rotateBoard, это видно на примере функции moveLeft:

```
undefined1 moveLeft(undefined8 param_1,undefined8 param_2)

{
  undefined1 uVar1;

  rotateBoard(param_1);
  uVar1 = moveUp(param_1,param_2);
  rotateBoard(param_1);
  rotateBoard(param_1);
  rotateBoard(param_1);
  return uVar1;
}
```

Рис. 11. moveLeft

Логика игры становится более понятной, но, возвращаясь к попытке поменять «2048» на «128», я пришел только к тому, что у нас нет функции выигрыша игры, если только случай проигрыша. Судить можно по отрывку main и gameEnded:

```
    if ((local_31 & 1) != 0) {
      drawBoard(auStack_28,local_32,local_30);
      usleep(150000);
      addRandom(auStack_28);
      drawBoard(auStack_28,local_32,local_30);
      uVar3 = gameEnded(auStack_28);
      if ((uVar3 & 1) != 0) {
        puts("         GAME OVER          ");
        goto LAB_00102418;
      }
    }
```

```
bool gameEnded(undefined8 param_1)

{
  bool bVar1;
  char cVar2;
  uint uVar3;

  cVar2 = countEmpty(param_1);
  if (cVar2 == '\0') {
    uVar3 = findPairDown(param_1);
    if ((uVar3 & 1) == 0) {
      rotateBoard(param_1);
      uVar3 = findPairDown(param_1);
      bVar1 = (uVar3 & 1) == 0;
      rotateBoard(param_1);
      rotateBoard(param_1);
      rotateBoard(param_1);
    }
    else {
      bVar1 = false;
    }
  }
  else {
    bVar1 = false;
  }
  return bVar1;
}
```

Рис. 12 и Рис. 13. Отрывок main и gameEnded

Логика примерно такая: если есть доступные ходы, мы продолжаем игру, если доступных ходов нет - «GAME OVER».

Тогда моя идея заменить «2048» на «128» для легкой победы не реализуется простым патчем. Облом.

Попробуем изменить арифметику игры! Сделаем так, чтобы при сложении квадратов «2» и «2» появился квадрат «2048».

Меня заинтересовала функция slideArray, вызываемая внутри moveUp.

```
2  undefined1 slideArray(long param_1,int *param_2)
3
4  {
5    byte bVar1;
6    undefined1 local_4;
7    byte local_3;
8    char local_2;
9
10   local_4 = 0;
11   local_2 = '\0';
12   for (local_3 = 0; local_3 < 4; local_3 = local_3 + 1) {
13     if (*(char *)(param_1 + (ulong)local_3) != '\0') {
14       bVar1 = findTarget(param_1,local_3,local_2);
15       if (bVar1 != local_3) {
16         if (*(char *)(param_1 + (ulong)bVar1) == '\0') {
17           *(undefined1 *)(param_1 + (ulong)bVar1) = *(undefined1 *)(param_1 + (ulong)local_3);
18         }
19         else if (*(char *)(param_1 + (ulong)bVar1) == *(char *)(param_1 + (ulong)local_3)) {
20           *(char *)(param_1 + (ulong)bVar1) = *(char *)(param_1 + (ulong)bVar1) + '\x01';
21           *param_2 = *param_2 + (1 << (ulong)(*(byte *)(param_1 + (ulong)bVar1) & 0x1f));
22           local_2 = bVar1 + 1;
23         }
24         *(undefined1 *)(param_1 + (ulong)local_3) = 0;
25         local_4 = 1;
26       }
27     }
28   }
29   return local_4;
30 }
```

Рис. 14. slideArray

Как раз она отвечает за сдвиг и сложение квадратов в одном ряду из четырех элементов! Здесь находится арифметика, которая нас интересует, а именно 19-20 строчки: при условии, что квадраты равны, значение текущего квадрата увеличивается на одну степень двойки, а второй квадрат обнуляется и соответственно исчезает. Также на 21 строчке прописано изменение общего счета игрока.

Увеличить значение на одну степень двойки - это «+’\x01’». В листинге видим интересующую нас инструкцию:

```
001011e8 20 00 00 8b     add        x0,x1,x0
001011ec 01 00 40 39     ldrb       w1,[x0]
001011f0 21 04 00 11     add        w1,w1,#0x1
001011f4 21 1c 00 12     and        w1,w1,#0xff
001011f8 01 00 00 39     strb       w1,[x0]
```

Рис. 15. Отрывок Listing для slideArray

Нас интересует инструкция по адресу 001011f0: add w1,w1,#0x1. Будем прибавлять не #0x1, а #0xA, или же 10 (если в 10-сс):

```
001011f0 21 28 00 11     add        w1,w1,#0xa
```

Мы запатчили игру. Экспортируем (конечно, сохранив исходную игру)!

```
-rw-r----- 1 parallels parallels 71680 Nov 12 21:15 patched
```

Рис. 16. Файл patched с правами доступа

После экспорта нам нужно дать себе разрешение на исполнение файла с помощью chmod +x:

```
parallels@panda:~/Desktop/re-lab/re-writeups$ chmod +x patched
parallels@panda:~/Desktop/re-lab/re-writeups$ ls -l
total 76
drwxrwxr-x 3 parallels parallels  4096 Nov 10 17:06 2048.c
-rwxr-x--x 1 parallels parallels 71680 Nov 12 21:15 patched
```

Рис. 17. patched с новыми правами для владельца rwx

Запускаем!

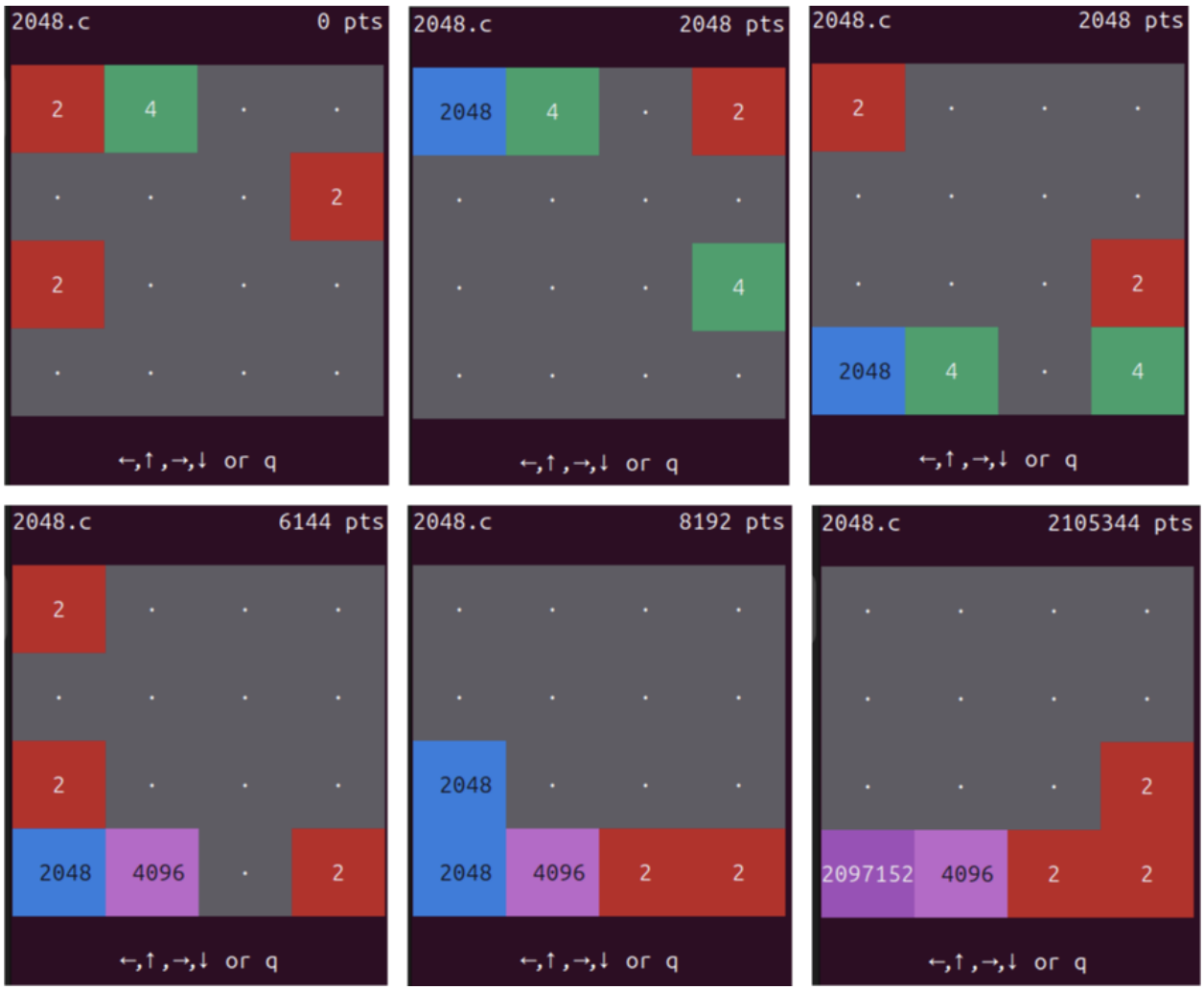


Рис. 18. Пример запатченной игры

Патч сработал! Как можем наблюдать, при сложении плиток с номиналом «2» мы получаем «2048», при сложении плиток с номиналом «4» получаем «4096». Грубо говоря, 2^1 + 2^1 станет равным 2^11, 2^2 + 2^2 станет равным 2^12 и т.д. Но что будет, если поиграть еще немного?

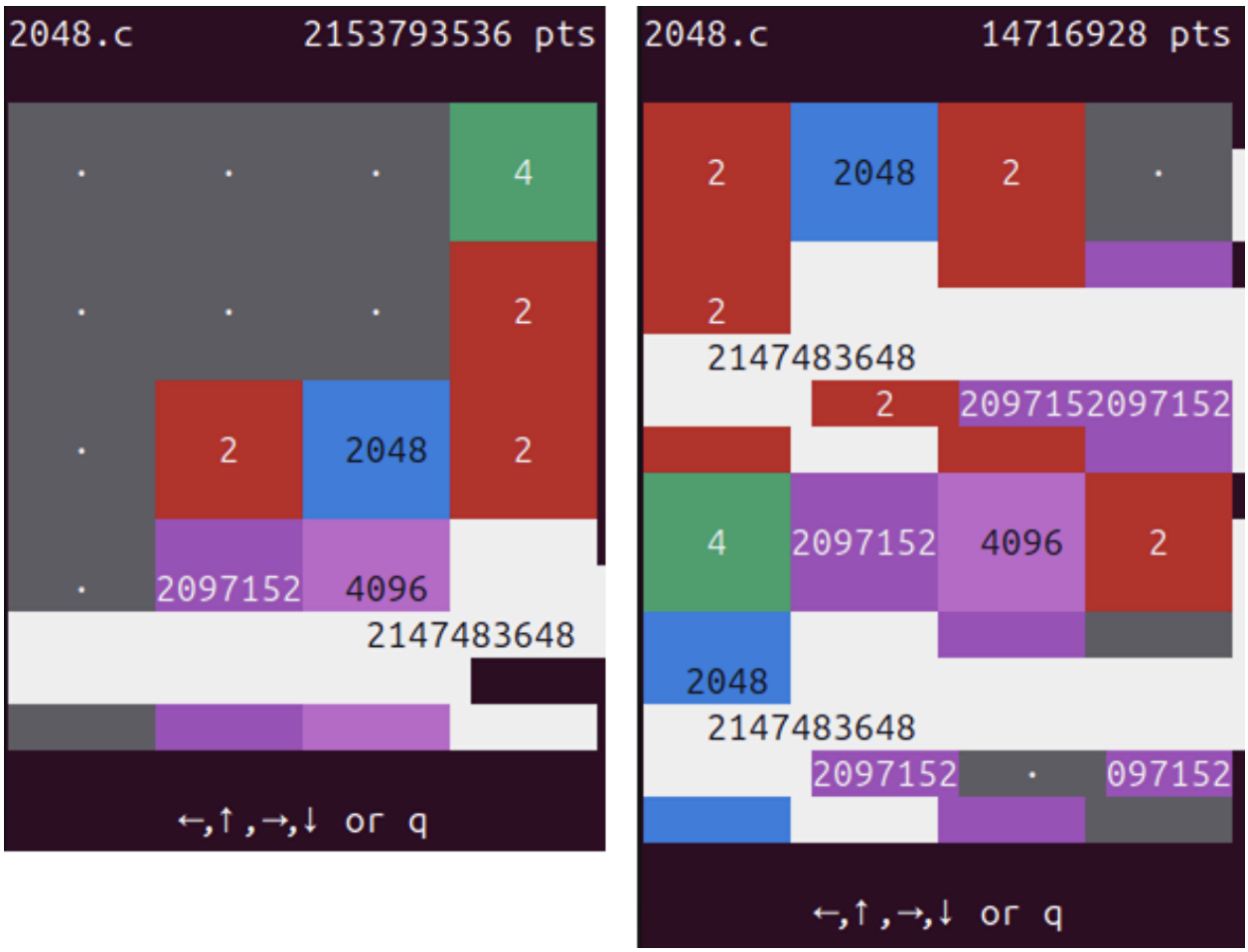


Рис. 19. Пример запатченной игры

Получатся слишком большие числа, из-за этого поле начинает ломаться и становится невозможно играть. Более того, наш счет становится меньше: из счета «2153793536» мы пришли к счету «14716928». Так происходит из-за фиксации на ширину клетки (в клетку помещается максимум 7 цифр - это видно и визуально, и можно подкрепить фрагментом функции drawBoard), а также из-за возможного 32-битного переполнения, т.к. счет хранится типом int (Рис. 20).

```
36          local_1c = 1 << (ulong)(*(byte *)(param_1 + (ulong)local_1f * 4 + (long)(int)(uint)local_1e)
37                                  & 0x1f);
38          cVar1 = getDigitCount(local_1c);
39          local_1d = 7 - cVar1;
40          printf("%*s%u%*s",(uint)local_1d - (uint)(local_1d >> 1),"",local_1c,(uint)(local_1d >> 1),
41                 "");
42        }
43        printf("\x1b[m");
```

Рис. 20. Отрывок drawBoard